

Artificial Neural Network For Regression Using PyTorch

Task: Regression Using ANN

Introduction

Artificial Neural Networks (ANNs) are a type of machine learning model inspired by the structure of biological neural networks. They are widely used for solving both classification and regression problems. In this project, a feedforward Artificial Neural Network was implemented using the PyTorch deep learning framework to solve a regression problem.

The dataset used in this project is the Bike Sharing Dataset, which contains information about hourly bike rentals and environmental conditions. The goal of the model is to predict the number of bike rentals (**cnt**) based on several input features such as temperature, humidity, and time-related variables.

The implementation includes data preprocessing, dataset scaling, neural network architecture design, model training, and evaluation.

Dataset Description

The dataset used in this project is the Bike Sharing Dataset(hour.csv and day.csv), specifically the **hour.csv** file. This dataset contains hourly records of bike rental usage along with environmental and seasonal information.

Some columns were removed because they were not needed for the regression task. The columns removed are instant, dteday, causal, registered.

The target variable is cnt - total number of bike rentals.

Features : season, yr, mnth, hr, holiday, weekday, workingday, weathersit, temp, atemp, hum, windspeed.

Data Preprocessing

Data preprocessing is an important step in machine learning. It ensures that the dataset is clean and properly formatted before training the model.

The following preprocessing steps were applied:

1. Loading the dataset using pandas.
2. Removing unnecessary columns.
3. Randomly shuffling the dataset.
4. Splitting the dataset into training, validation, and testing sets.
5. Feature scaling using StandardScaler.

Dataset split: Training set (60%), Validation set (20%) and Test set (20%)

Feature Scaling

Feature scaling is required because different features may have different ranges. Neural networks perform better when inputs are normalized.

In this project, **StandardScaler** was used. It standardizes features using the formula:

$$X_{\text{scaled}} = (X - \mu) / \sigma$$

Where:

μ = mean of the feature

σ = standard deviation

The scaler is **fitted only on the training data** and then applied to validation and test datasets. This prevents data leakage.

Conversion to PyTorch Tensors

Since the neural network was implemented using PyTorch, the NumPy arrays were converted into PyTorch tensors. Tensors are the fundamental data structure used by PyTorch for numerical computations and automatic differentiation.

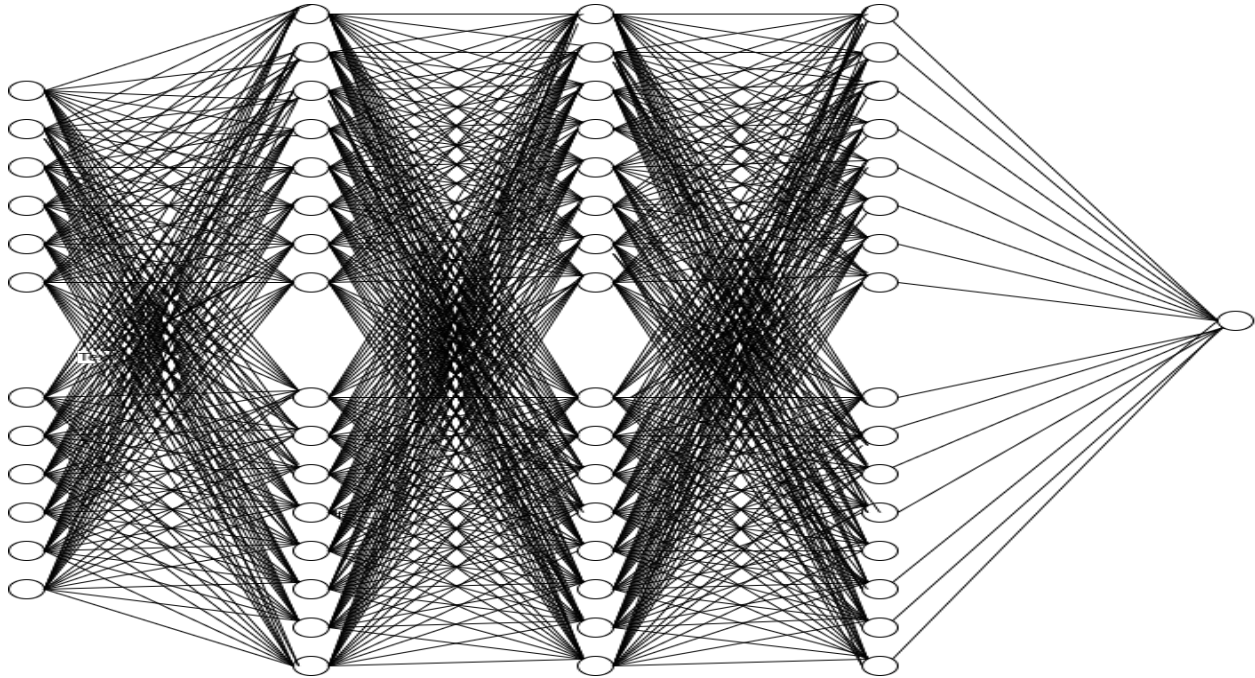
```
X_train = torch.tensor(x_train, dtype=torch.float32)
```

Artificial Neural Network Architecture

The neural network consists of three hidden layers. Each hidden layer uses the ReLU activation function to introduce non-linearity. The output layer contains a single neuron because this is a regression problem where the model predicts a continuous ANN Structure.

ANN Structure: Input Layer(12 features), Hidden Layer 1 (64 neurons), Hidden Layer 2 (32 neurons), Hidden Layer 3 (16 neurons) and Output Layer (1 neuron)

Activation function: ReLU



Loss Function

The loss function used in this project is Mean Squared Error (MSE). It measures the average squared difference between predicted values and true values.

Formula:

$$\text{MSE} = (1/n) \sum (y_{\text{pred}} - y_{\text{true}})^2$$

MSE is commonly used for regression problems because it penalizes large errors more heavily.

Optimizer

The optimizer used in this project is **Adam (Adaptive Moment Estimation)**. Adam is an efficient gradient-based optimization algorithm that adapts the learning rate during training.

```
Optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Learning rate used : 0.001

Training Process

The model was trained for 200 epochs. During each epoch the following steps occur:

1. Forward pass
2. Loss computation
3. Backpropagation
4. Weight update using the optimizer

Validation loss is also computed after each epoch to monitor the model's performance on unseen data.

Model Evaluation

After training, the model was evaluated using the test dataset. The following metrics were calculated:

MSE – Mean Squared Error

RMSE – Root Mean Squared Error

MAE – Mean Absolute Error

These metrics measure how close the model predictions are to the actual values.

Conclusion:

In this project, an Artificial Neural Network was implemented to solve a regression problem using PyTorch. The model successfully learned relationships between environmental features and bike rental counts. Data preprocessing, feature scaling, and proper dataset splitting were essential for achieving stable training.

This project demonstrates how deep learning models can be applied to real-world regression problems.